
Systemes Embarqués



ROB 2A, UE 4.1, ENSTA

Introduction à la conteneurisation

SYSTÈMES EMBARQUÉS

But it works on my machine

Situation où un logiciel, un script ou une application fonctionne parfaitement **sur son propre environnement de développement...**

... mais échoue ou se comporte différemment **sur la machine d'un collègue, en production, ou dans un autre environnement !**

Un environnement de développement rassemble OS, librairies, déclarations des dépendances, permissions, variables d'environnement, ...

Autant de paramètres qui peuvent différer d'un environnement à l'autre et causer un dysfonctionnement



<https://programmerhumor.io>

Qu'est ce que la conteneurisation ?

Packaging d'un logiciel avec un système d'exploitation minimal, des bibliothèques et des outils pour exécuter le code ou l'application sous forme d'exécutable léger, appelé conteneur

- Le conteneur peut s'exécuter de manière cohérente sur n'importe quelle infrastructure, sans procédure d'installation spécifique
- Le conteneur partage le système d'exploitation hôte, solution plus légère qu'une machine virtuelle (VM)
- Applications « écrites une fois et exécutées partout » (grande portabilité, installation simplifiée...)

from : <https://www.ibm.com/cloud/learn/containerization>

Container VS Virtual Machine

Virtual Machine

Une machine virtuelle (VM) est un système d'exploitation complet doté de son propre noyau, de ses drivers matériels, de ses programmes et de ses applications.

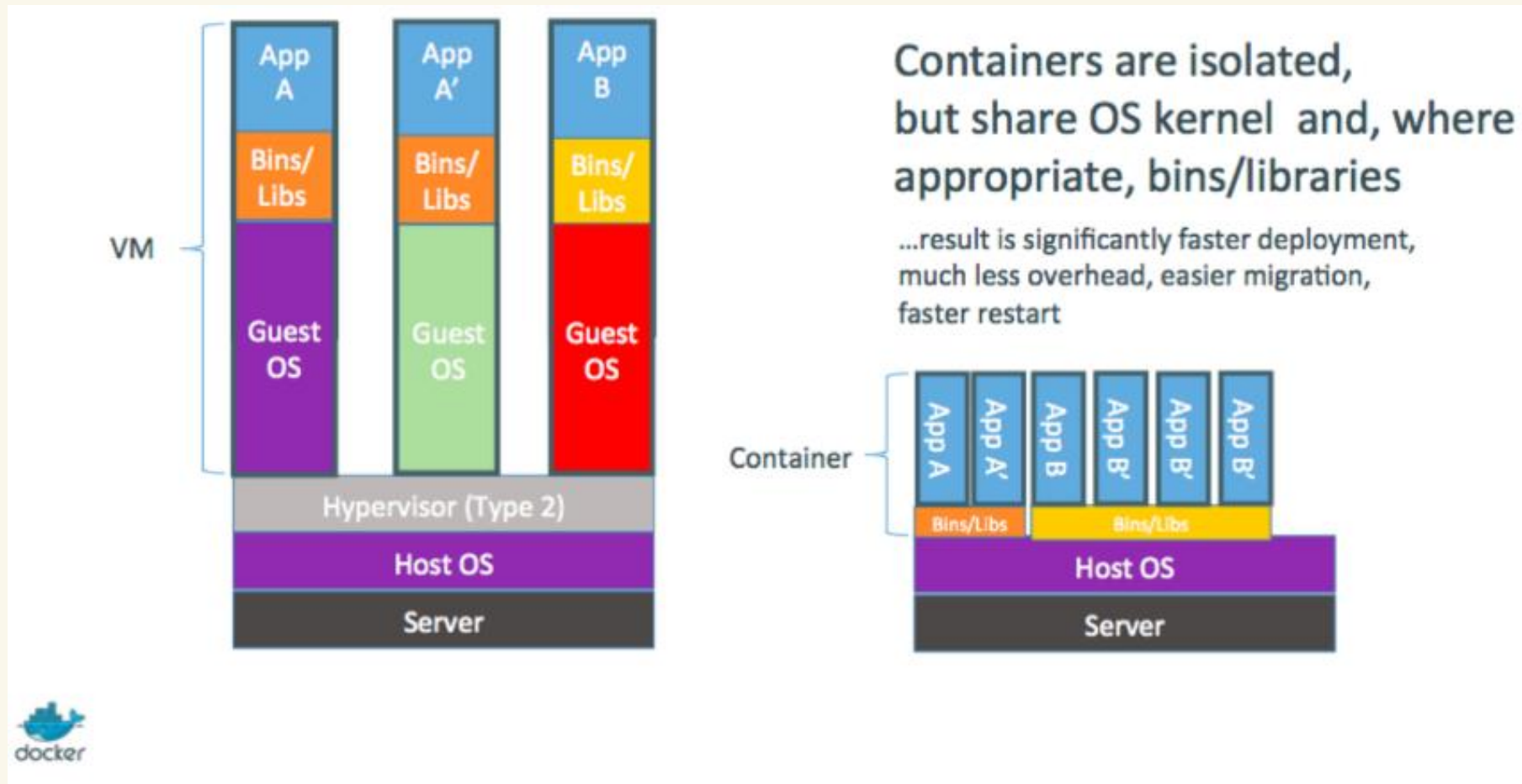
Créer une VM uniquement pour isoler une seule application représente une charge importante.

Container

Un conteneur est simplement un processus isolé contenant tous les fichiers nécessaires à son exécution.

Si vous exécutez plusieurs conteneurs, ils partagent le même noyau, ce qui vous permet d'exécuter davantage d'applications avec une infrastructure réduite.

Container VS Virtual Machine



Container VS Virtual Machine

Un conteneur n'est pas une machine virtuelle !

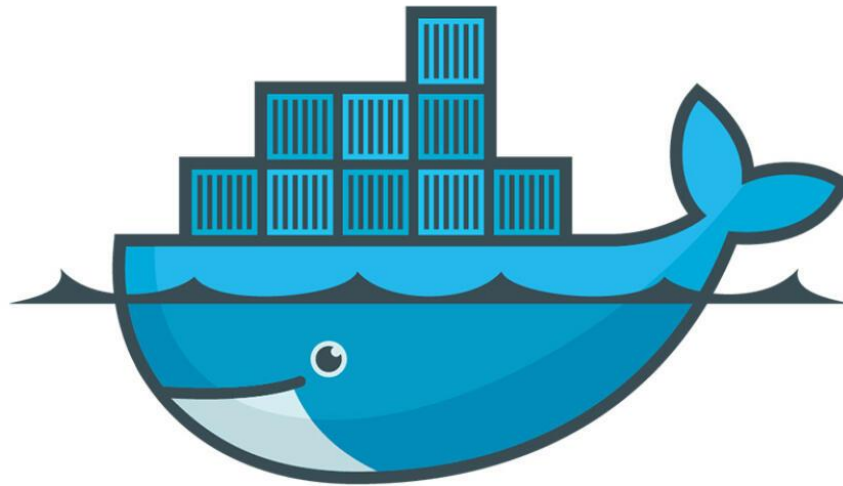
Mais il peut parfois être utilisé comme une sorte de VM (par exemple pour tester des programmes sur un système d'exploitation différent ou pour exécuter un petit serveur web isolé de l'ordinateur hôte).

Différence importante :

un conteneur partage le système d'exploitation de l'ordinateur hôte, contrairement à une VM qui peut exécuter un système d'exploitation différent.

Notez que l'exécution d'un système d'exploitation différent prend plus de temps que le partage du système d'exploitation existant.

Docker: an open-source containerizer



<https://www.docker.com/>

Qu'est ce qu'une image de conteneur ?

Une image est un package standardisé qui inclut tous les fichiers, binaires, bibliothèques et configurations nécessaires à l'exécution d'un conteneur.

Exemple : Une image ubuntu:20.04 est une copie figée d'Ubuntu 20.04 avec ses outils de base.

Caractéristiques :

- **Immuable** : On ne la modifie pas directement (on en crée une nouvelle version si besoin).
- **Partageable** : On peut la stocker sur [Docker Hub](#) ou un registre privé.
- **Basée sur des couches** : Chaque instruction dans un Dockerfile ajoute une couche à l'image.

Lien entre image et conteneur

Un **conteneur** est une **instance exécutable** d'une image.

C'est un processus isolé qui tourne sur la machine, avec :

- Son propre système de fichiers (issu de l'image).
- Ses propres ressources (CPU, mémoire, réseau).
- Une isolation par rapport au système hôte et aux autres conteneurs.

Caractéristiques :

- **Éphémère** : Par défaut, les changements dans un conteneur (fichiers créés, logs) disparaissent quand il s'arrête (sauf si on utilise des **volumes** ou -v).
- **Isolé** : Chaque conteneur a son propre espace réseau, processus, etc.

Docker Hub



Docker Hub est la plateforme mondiale de référence pour le stockage et la distribution d'images. Elle propose plus de 100 000 images créées par des développeurs et exécutables localement.

Exemple: Debian, Ubuntu, ros images

Docker Engine

Moteur de base de Docker, un logiciel qui permet de **créer, gérer et exécuter des conteneurs**.

=> **Docker Daemon (dockerd)** :

Processus qui gère les objets Docker (images, conteneurs, réseaux, volumes).

Écoute les commandes de l'API Docker (ex : docker run).

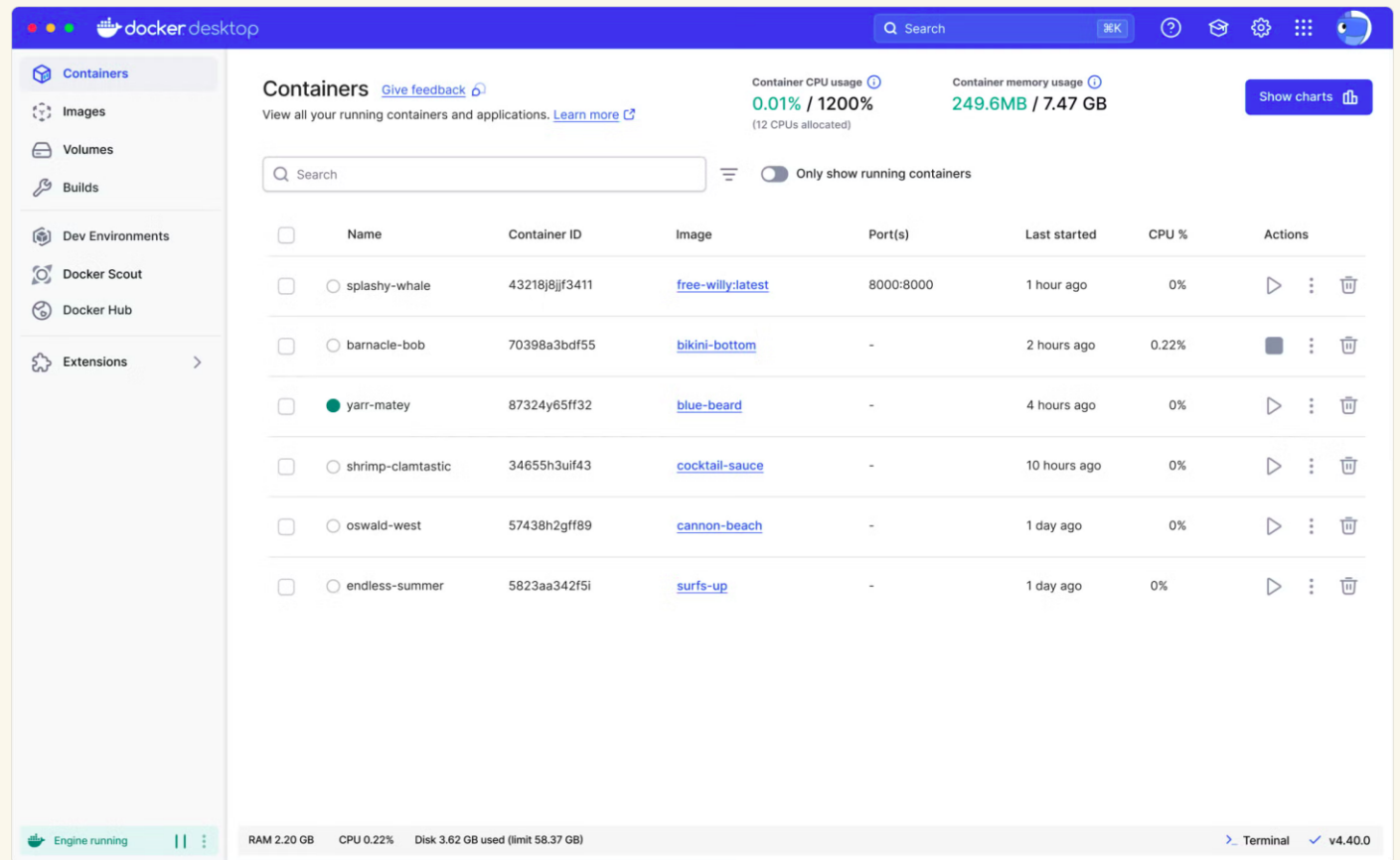
=> **CLI (docker)** : Ligne de commande (docker build, docker run, etc.) qui envoie des instructions au daemon.

- **Indispensable** pour utiliser Docker.
- Disponible sur **Linux** (nativement) et **Windows/macOS** (via Docker Desktop).
- Open source et gratuit.

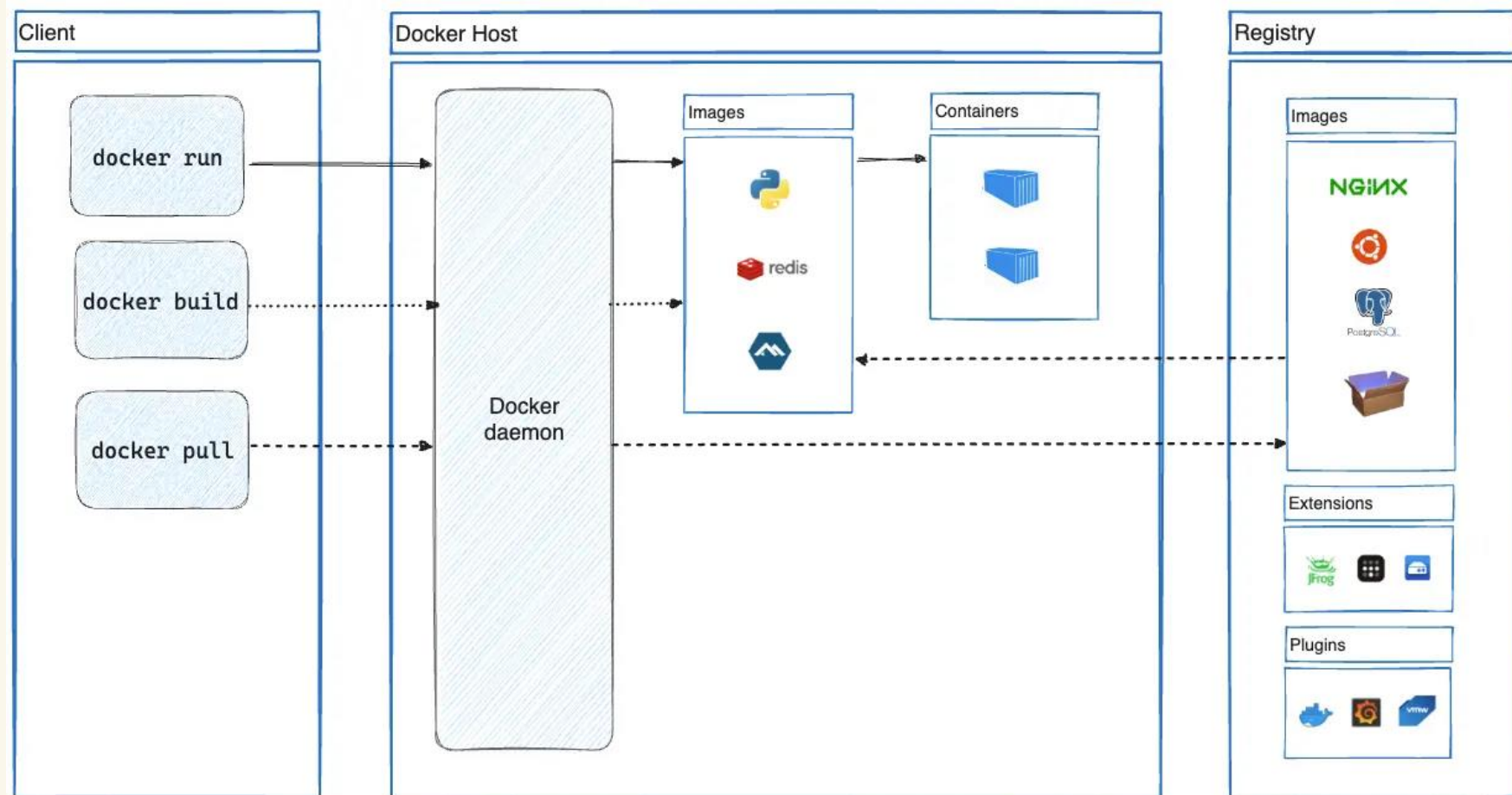
Docker Desktop

Une **application graphique** (GUI) qui simplifie l'utilisation de Docker sur **Windows** et **macOS**.

Elle embarque Docker Engine et ajoute des outils visuels pour gérer les conteneurs, images, etc.



Docker architecture



<https://www.docker.com/>

Objectifs

L'un des cas d'applications d'un conteneur est d'exécuter des logiciels avec différentes versions de systèmes d'exploitation.

Exemples : sur un ordinateur Ubuntu 22.04, exécuter ROS2 Foxy (qui ne fonctionne que sur Ubuntu 20.04) ou ROS1 Kinetic (qui ne fonctionne que sur Ubuntu 16.04).

Dans ce cours d'introduction nous allons :

- Obtenir et modifier des images
- Build des images avec un Dockerfile
- Utiliser des conteneurs
- ...

=> Lab 4 – Getting started with Docker